



Lab 07 – Worksheet

Name: Maleeha Prabha

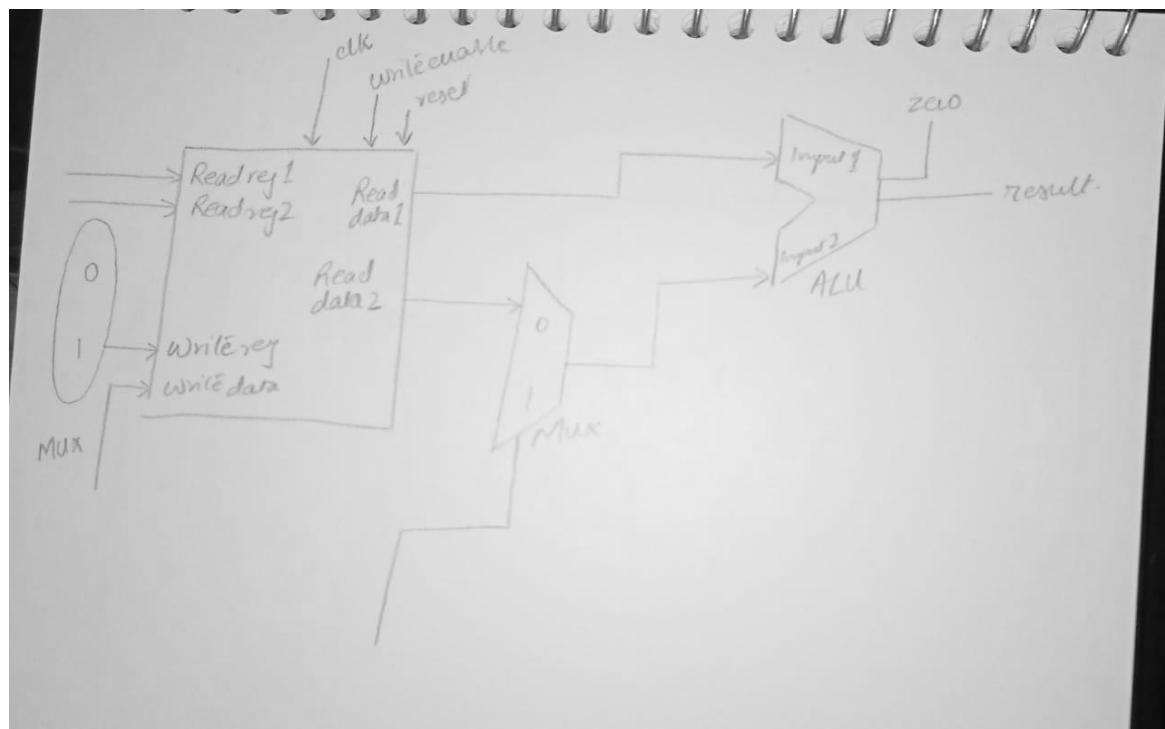
ID: 09991
10292

Section: T1

Note: Assumptions and logics should be explained separately in tasks after the task results.

Task 1

Block Diagrams





Comments

Task 2

Modules + Testbench:

```
RegisterFile
`timescale 1ns / 1ps
/////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 03/05/2026 09:08:32 AM
// Design Name:
// Module Name: RegisterFile
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
/////////////////////////////////////////////////////////////////

module RegisterFile(
    input clk, rst, WriteEnable,
    input [4:0] rs1, rs2, rd,
    input [31:0] WriteData,
    output [31:0] readData1, readData2
);
```



```
reg[31:0] regs [31:0];
initial begin
    regs[0] = 32'h00000000;
    regs[1] = 32'h10101010;
    regs[2] = 32'h01010101;
end

always @(posedge clk) begin
    if (WriteEnable == 1 && rd != 5'b0) regs[rd] <= WriteData;
end

assign readData1=regs[rs1];
assign readData2=regs[rs2];

endmodule

register file testbench
`timescale 1ns / 1ps

module RegisterFile_tb;

    reg clk;
    reg reset;
    reg WriteEnable;
    reg [4:0] rs1;
    reg [4:0] rs2;
    reg [4:0] rd;
    reg [31:0] WriteData;
    wire [31:0] readData1;
    wire [31:0] readData2;

    // Instantiate DUT (Device Under Test)
    RegisterFile uut (
        .clk(clk),
        .reset(reset),
        .WriteEnable(WriteEnable),
        .rs1(rs1),
        .rs2(rs2),
        .rd(rd),
        .WriteData(WriteData),
        .readData1(readData1),
        .readData2(readData2)
    );
```



```
always #5 clk = ~clk;
initial begin

    $display("Starting Register File Testbench");
    //test1
    clk = 0;
    reset = 1;
    WriteEnable = 0;
    rs1 = 0;
    rs2 = 0;
    rd = 0;
    WriteData = 0;
    #10;
    reset = 0;
    rs1 = 5;
    #10;
    $display("After reset, x5 = %h (should be 0)", readData1);
    //test2
    rd = 5;
    WriteData = 32'hDEADBEEF;
    WriteEnable = 1;
    #10;
    WriteEnable = 0;
    rs1 = 5;
    #10;

    $display("Write x5 = %h (expect DEADBEEF)", readData1);

    //test3
    rd = 0;
    WriteData = 32'hFFFFFFFF;
    WriteEnable = 1;
    #10;
    WriteEnable = 0;
    rs1 = 0;
    #10;
    $display("x0 after write attempt = %h (should be 0)", readData1);

    //test4
    rd = 6;
    WriteData = 32'h12345678;
    WriteEnable = 1;
    #10;

    WriteEnable = 0;

    rs1 = 5;
    rs2 = 6;
```



```
#10;

$display("Read rs1(x5) = %h (expect DEADBEEF)", readData1);
$display("Read rs2(x6) = %h (expect 12345678)", readData2);

//test5
rd = 5;
WriteData = 32'hAAAAAAAA;
WriteEnable = 1;
#10;
WriteEnable = 0;
rs1 = 5;
#10;
$display("Overwrite x5 = %h (expect AAAAAAAAA)", readData1);
#20;
$display("Testbench complete.");
$finish;

end

endmodule

ALU
`timescale 1ns / 1ps
/////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 02/26/2026 09:43:45 AM
// Design Name:
// Module Name: ALU_module
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
/////////////////////////////////////////////////////////////////

module ALU(
    input [31:0]A, input [31:0]B,
    input [3:0]ALUControl,
```



```
output reg [31:0]ALUResult,  
output zero  
);
```

```
always @(*) begin
```

```
    if (ALUControl == 4'b0000) ALUResult <= A+B;  
    else if (ALUControl == 4'b0001) ALUResult <= A-B;  
    else if (ALUControl == 4'b0010) ALUResult <= A&B;  
    else if (ALUControl == 4'b0011) ALUResult <= A|B;  
    else if (ALUControl == 4'b0100) ALUResult <= A^B;  
    else if (ALUControl == 4'b0101) ALUResult <= A<<B[4:0];  
    else if (ALUControl == 4'b0110) ALUResult <= A>>B[4:0];  
    else ALUResult <= 32'b0;
```

```
end
```

```
assign zero = (ALUResult==0);
```

```
endmodule
```

RF ALU FSM Testbench

```
`timescale 1ns/1ps
```

```
module RF_ALU_FSM_tb;
```

```
reg clk;
```

```
reg rst;
```

```
reg WriteEnable;
```

```
reg [4:0] rs1, rs2, rd;
```

```
reg [31:0] WriteData;
```

```
reg [3:0] ALUControl;
```

```
wire [31:0] ReadData1;
```

```
wire [31:0] ReadData2;
```

```
wire [31:0] ALUResult;
```

```
wire Zero;
```

```
////////////////////////////////////
```

```
// Instantiate Register File
```

```
////////////////////////////////////
```

```
RegisterFile RF_inst(
```

```
    .clk(clk),
```

```
    .reset(rst),
```

```
    .WriteEnable(WriteEnable),
```

```
    .rs1(rs1),
```



```
.rs2(rs2),
.rd(rd),
.WriteData(WriteData),
.readData1(ReadData1),
.readData2(ReadData2)
);

////////////////////////////////////
// Instantiate ALU
////////////////////////////////////

ALU_module ALU_inst(
.A(ReadData1),
.B(ReadData2),
.ALUControl(ALUControl),
.ALUResult(ALUResult),
.zero(Zero)
);

////////////////////////////////////
// FSM Variables
////////////////////////////////////

reg [2:0] current_state;
reg [2:0] alu_step;
reg [1:0] write_count;

parameter START      = 3'd0;
parameter INIT_WRITE = 3'd1;
parameter FETCH_REGS = 3'd2;
parameter EXECUTE_ALU = 3'd3;
parameter STORE_RES  = 3'd4;

////////////////////////////////////
// Clock generation
////////////////////////////////////

always #5 clk = ~clk;

////////////////////////////////////
// FSM Logic
////////////////////////////////////

always @(posedge clk or posedge rst)
begin

if(rst)
begin
```



```
current_state <= START;
alu_step <= 0;
write_count <= 0;
end

else
begin

case(current_state)

//////////////////////////
// Idle state
//////////////////////////

START:
begin
    WriteEnable <= 0;
    current_state <= INIT_WRITE;
end

//////////////////////////
// Initialize registers
//////////////////////////

INIT_WRITE:
begin
    WriteEnable <= 1;

    case(write_count)

        2'd0:
        begin
            rd <= 5'd1;
            WriteData <= 32'hAAAA0000;
        end

        2'd1:
        begin
            rd <= 5'd2;
            WriteData <= 32'h0000BBBB;
        end

        2'd2:
        begin
            rd <= 5'd3;
            WriteData <= 32'h00000008;
        end
```




```
////////////////////////////////////
```

```
STORE_RES:
```

```
begin
```

```
    WriteEnable <= 1;
```

```
    rd <= 5'd6 + alu_step;
```

```
    WriteData <= ALUResult;
```

```
    WriteEnable <= 1;
```

```
    if(alu_step == 3'd6)
```

```
    begin
```

```
        // BEQ-style check
```

```
        rs1 <= 5'd1;
```

```
        rs2 <= 5'd1;
```

```
        ALUControl <= 4'b0001;
```

```
        if(Zero)
```

```
        begin
```

```
            rd <= 5'd15;
```

```
            WriteEnable <= 1;
```

```
            WriteData <= 32'h00000001;
```

```
        end
```

```
        $finish;
```

```
    end
```

```
    else
```

```
    begin
```

```
        alu_step <= alu_step + 1;
```

```
        current_state <= FETCH_REGS;
```

```
    end
```

```
end
```

```
endcase
```

```
end
```

```
end
```

```
////////////////////////////////////
```

```
// Initial block
```

```
////////////////////////////////////
```

```
initial
```

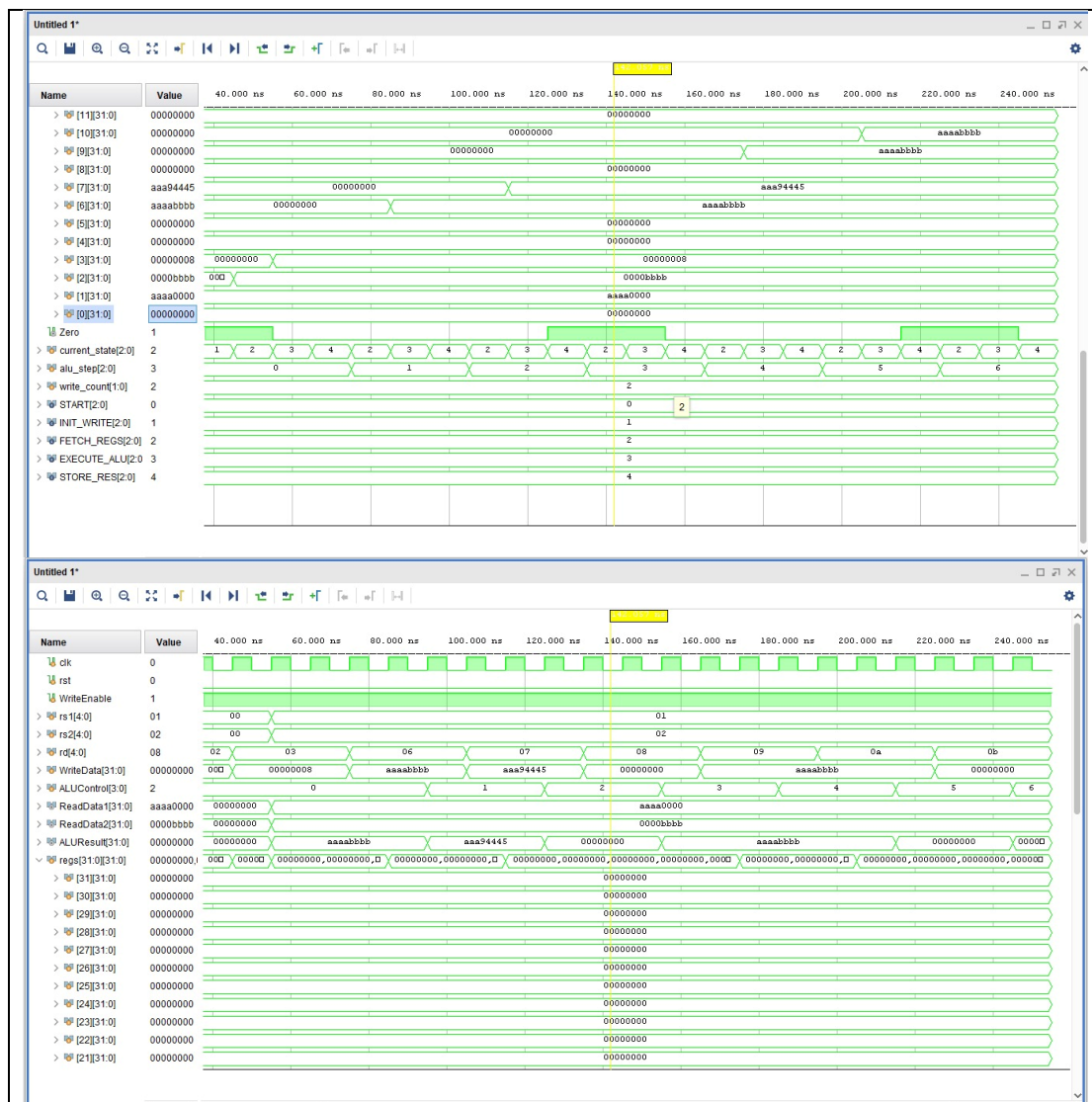
```
begin
```

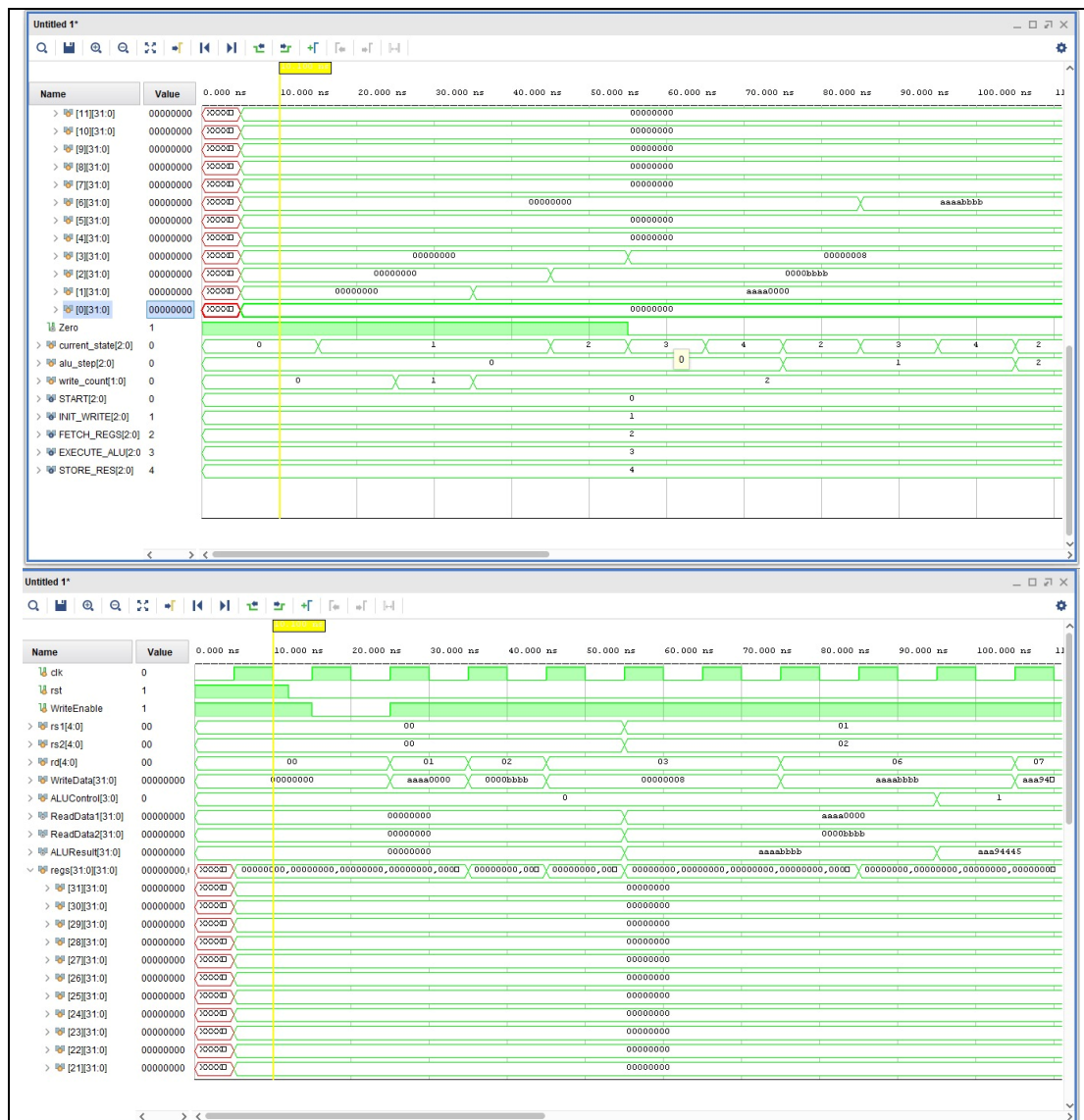
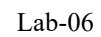


```
clk = 0;  
rst = 1;  
  
WriteEnable = 1;  
rs1 = 0;  
rs2 = 0;  
rd = 0;  
WriteData = 0;  
ALUControl = 0;  
  
#12 rst = 0;  
  
end  
  
endmodule
```



Results:







Task 3

RTL Schematics

**Top module**

```
`timescale 1ns / 1ps

module top_rf_alu (
    input wire    clk,
    input wire [15:0] SW,
    input wire [4:0] BTN,
    output wire [15:0] LED
);

wire rst;

debounce db_inst (
    .clk (clk),
    .pbin (BTN[0] | SW[15]),
    .pbout(rst)
);

wire [31:0] sw_data;

switches sw_inst (
    .clk      (clk),
    .rst      (rst),
    .bns      ({11'b0, BTN}),
    .switches (SW),
    .readEnable (1'b1),
    .writeData  (32'b0),
    .writeEnable(1'b0),
    .readData   (sw_data)
);

reg    rf_we;
reg [4:0] rf_rs1;
reg [4:0] rf_rs2;
reg [4:0] rf_rd;
reg [31:0] rf_wdata;

wire [31:0] rf_out1;
wire [31:0] rf_out2;

RegisterFile RF_inst (
    .clk      (clk),
    .reset    (rst),
    .WriteEnable(rf_we),
    .rs1      (rf_rs1),
    .rs2      (rf_rs2),
```



```
.rd      (rf_rd),
.WriteData (rf_wdata),
.readData1 (rf_out1),
.readData2 (rf_out2)
);
reg [3:0] alu_ctrl;
wire [31:0] alu_res;
wire      alu_zero;

ALU_module ALU_inst (
    .A      (rf_out1),
    .B      (rf_out2),
    .ALUControl(alu_ctrl),
    .ALUResult (alu_res),
    .zero     (alu_zero)
);

reg [31:0] led_din;
wire [31:0] led_dout;
wire [15:0] led_out;

assign led_out = led_dout[15:0];
assign LED     = led_out;

leds led_inst (
    .clk      (clk),
    .rst      (rst),
    .btns     ({11'b0, BTN}),
    .writeData (led_din),
    .writeEnable(1'b1),
    .readEnable (1'b1),
    .memAddress (30'b0),
    .switches  (SW),
    .readData   (led_dout)
);

localparam [2:0]
    S_RESET  = 3'd0,
    S_WRITE_A = 3'd1,
    S_WRITE_B = 3'd2,
    S_FETCH  = 3'd3,
    S_EXECUTE = 3'd4,
    S_STORE  = 3'd5,
    S_BEQ    = 3'd6,
    S_IDLE   = 3'd7;

localparam [31:0] CONST_A = 32'h10101010;
localparam [31:0] CONST_B = 32'h01010101;
```



```
reg [2:0] state;
reg [2:0] op_idx;

reg [28:0] tick_cnt;
reg      tick;

always @(posedge clk) begin
    if (rst) begin
        tick_cnt <= 29'd0;
        tick     <= 1'b0;
    end else begin
        tick <= 1'b0;
        if (tick_cnt == 29'd299_999_999) begin // 3 seconds at 100 MHz
            tick_cnt <= 29'd0;
            tick     <= 1'b1;
        end else begin
            tick_cnt <= tick_cnt + 1;
        end
    end
end

always @(posedge clk) begin
    if (rst) begin
        state <= S_RESET;
        op_idx <= 3'd0;
        rf_we  <= 1'b0;
        rf_rs1 <= 5'd0;
        rf_rs2 <= 5'd0;
        rf_rd  <= 5'd0;
        rf_wdata <= 32'd0;
        alu_ctrl <= 4'b0000;
    end
    else if (tick) begin
        rf_we <= 1'b0;

        case (state)

            S_RESET: begin
                op_idx <= 3'd0;
                state <= S_WRITE_A;
            end

            // write x1 = 0x10101010
            S_WRITE_A: begin
                rf_we  <= 1'b1;
                rf_rd  <= 5'd1;
                rf_wdata <= CONST_A;
            end
        endcase
    end
end
```



```
state <= S_WRITE_B;
end

// write x2 = 0x01010101
S_WRITE_B: begin
    rf_we <= 1'b1;
    rf_rd <= 5'd2;
    rf_wdata <= CONST_B;
    state <= S_FETCH;
end

S_FETCH: begin
    rf_rs1 <= 5'd1;
    rf_rs2 <= 5'd2;
    state <= S_EXECUTE;
end

S_EXECUTE: begin
    case (op_idx)
        3'd0: alu_ctrl <= 4'b0000; // ADD
        3'd1: alu_ctrl <= 4'b0001; // SUB
        3'd2: alu_ctrl <= 4'b0010; // AND
        3'd3: alu_ctrl <= 4'b0011; // OR
        3'd4: alu_ctrl <= 4'b0100; // XOR
        3'd5: alu_ctrl <= 4'b0101; // SLL
        3'd6: alu_ctrl <= 4'b0110; // SRL
        default: alu_ctrl <= 4'b0000;
    endcase
    state <= S_STORE;
end

S_STORE: begin
    rf_we <= 1'b1;
    rf_rd <= 5'd4 + {2'b00, op_idx};
    rf_wdata <= alu_res;

    if (op_idx == 3'd6) begin
        state <= S_BEQ;
    end else begin
        op_idx <= op_idx + 1;
        state <= S_FETCH;
    end
end

S_BEQ: begin
    rf_rs1 <= 5'd1;
    rf_rs2 <= 5'd1;
    alu_ctrl <= 4'b0001; // SUB
```



```
        state  <= S_IDLE;
    end

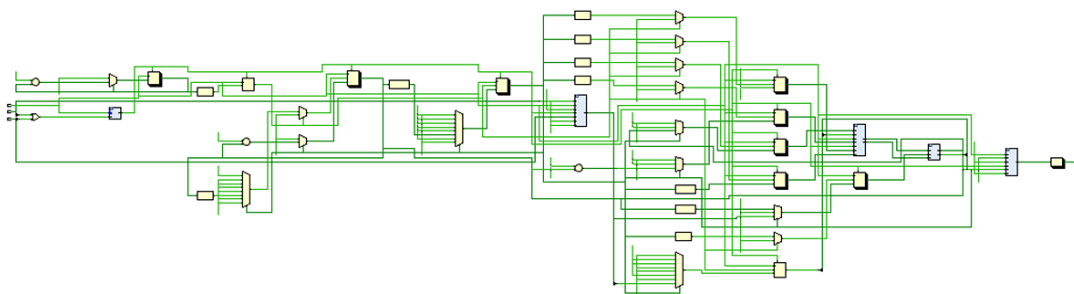
    S_IDLE: begin
        rf_rs1  <= 5'd1;
        rf_rs2  <= 5'd2;
        alu_ctrl <= {1'b0, sw_data[2:0]};
        rf_we   <= sw_data[3];
        state   <= S_RESET;
    end

    default: state <= S_RESET;

endcase
end
end

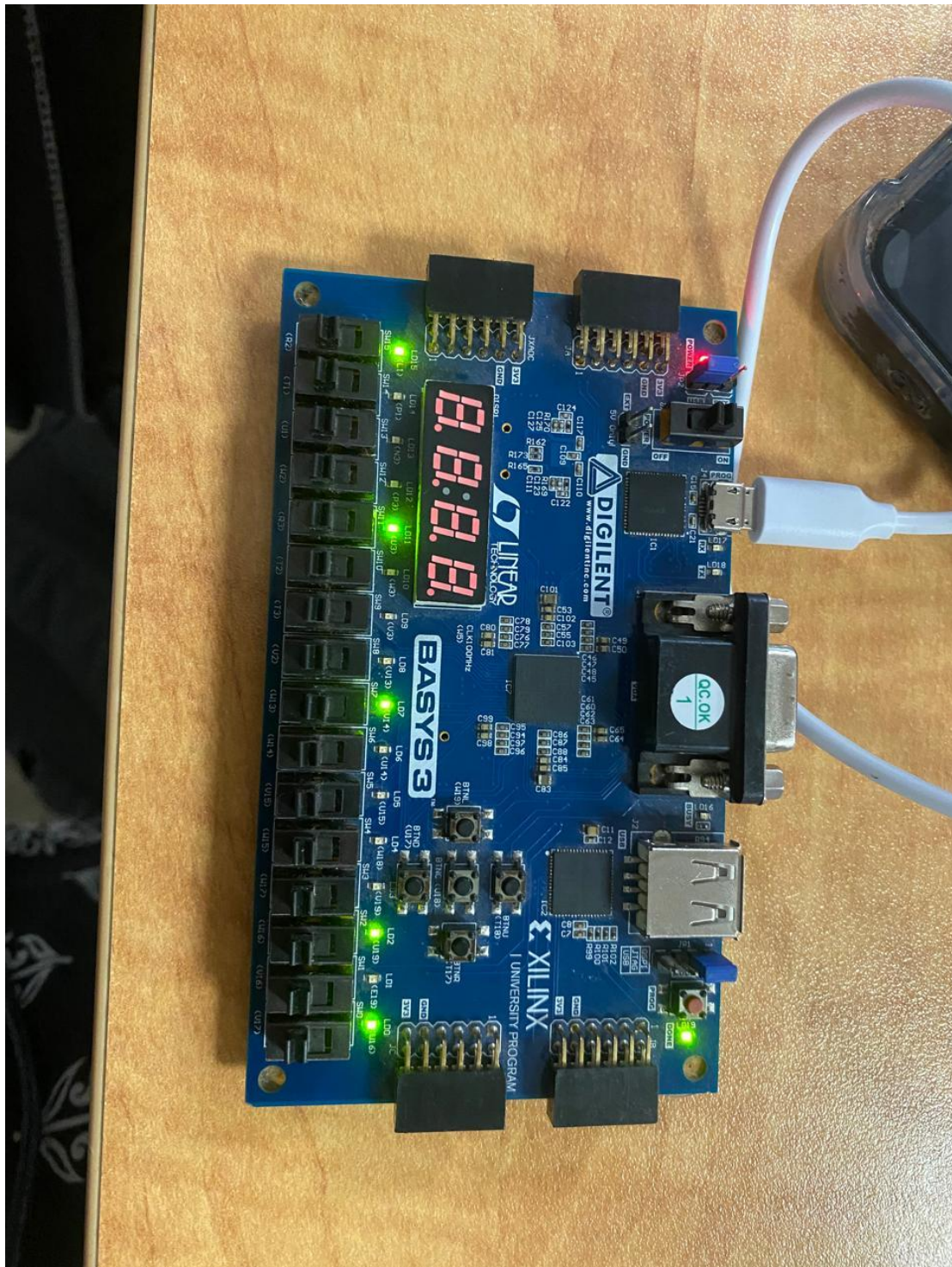
always @(*) begin
    led_din = {
        16'b0,
        alu_res[8:0], // LED[15:7] result
        alu_zero,     // zero -led6
        op_idx,       // LED[5:3] operation
        state         // LED[2:0] fsm
    };
end

endmodule
```





Add





Sub



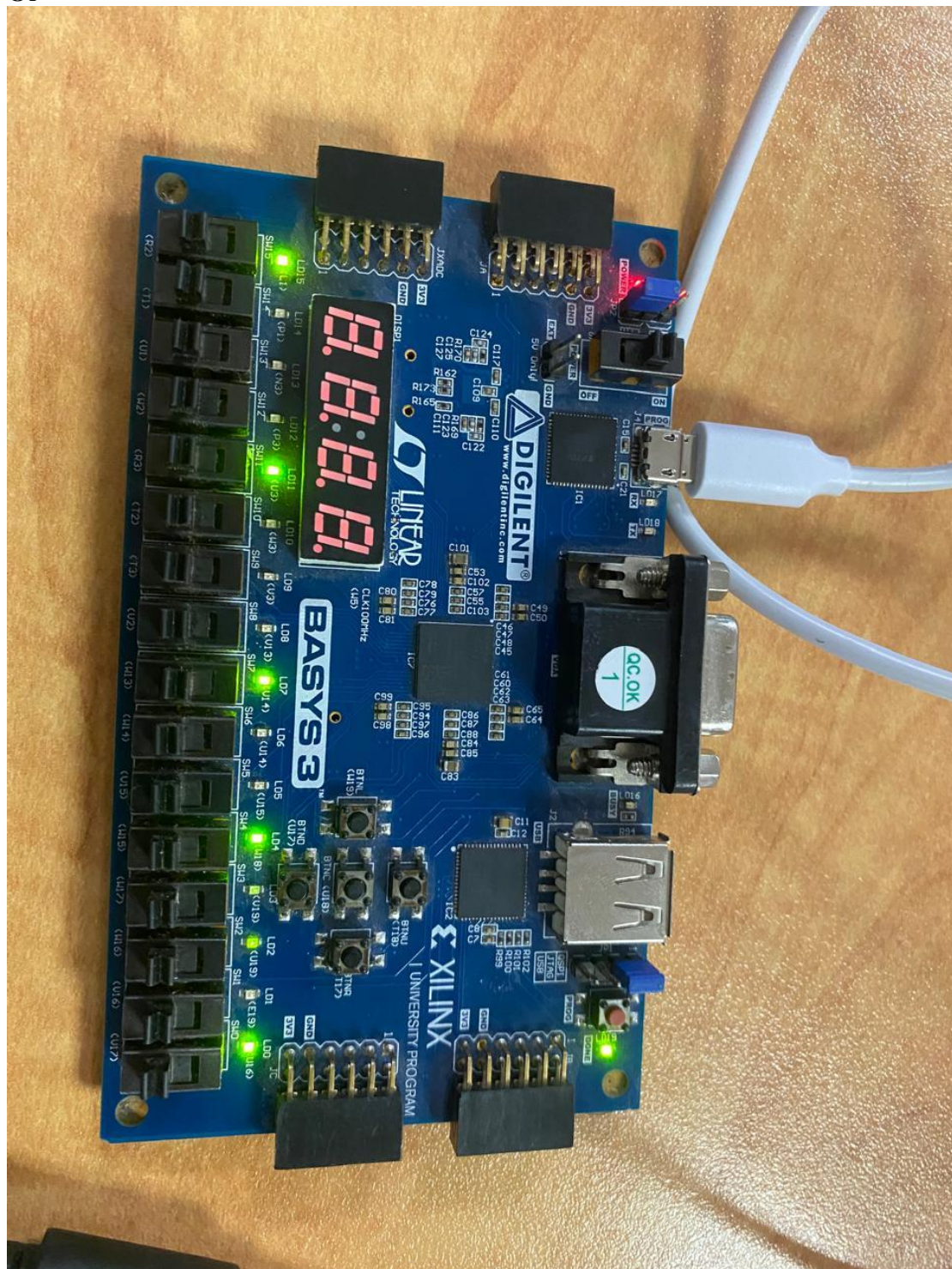


And



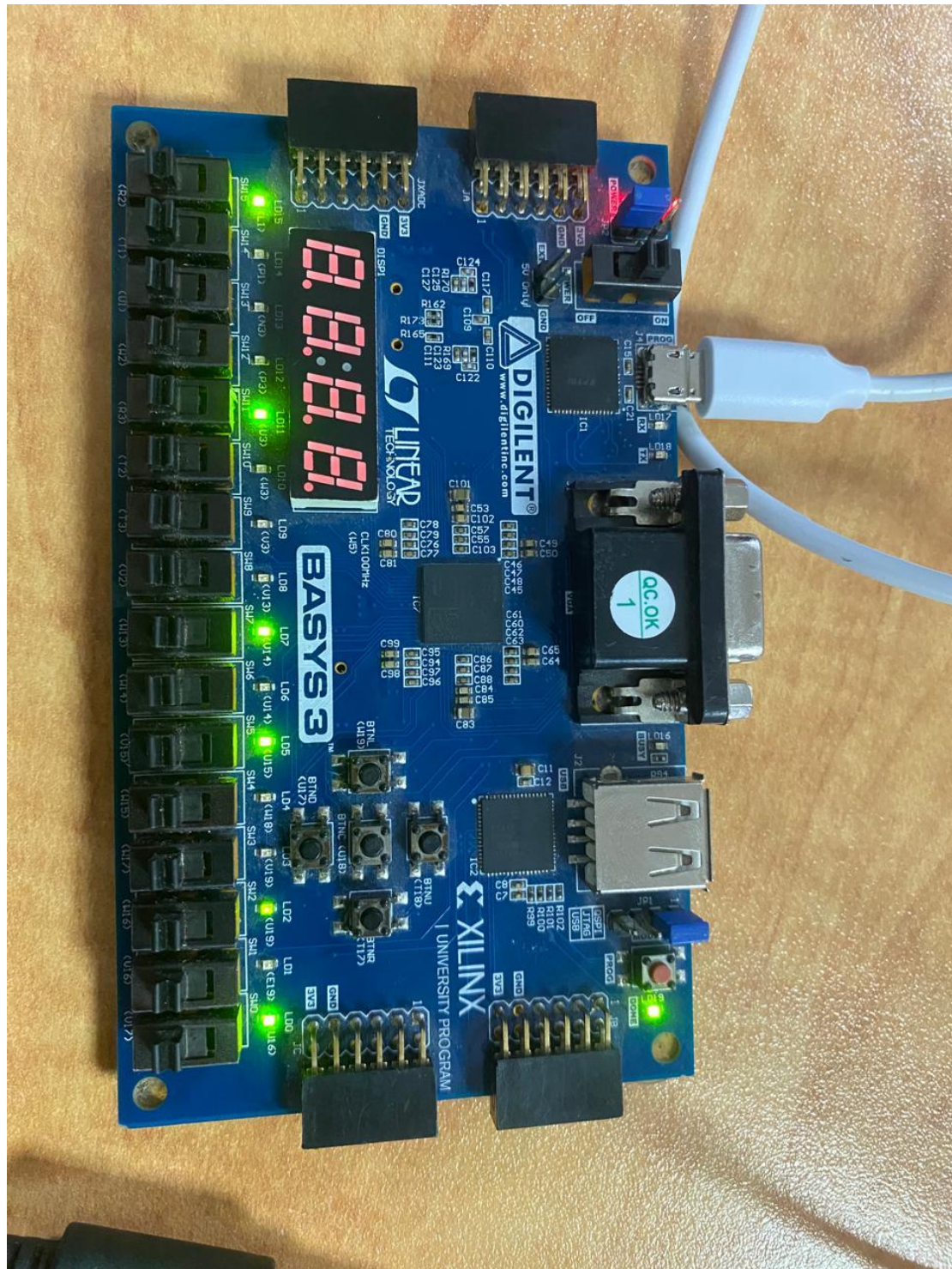


Or

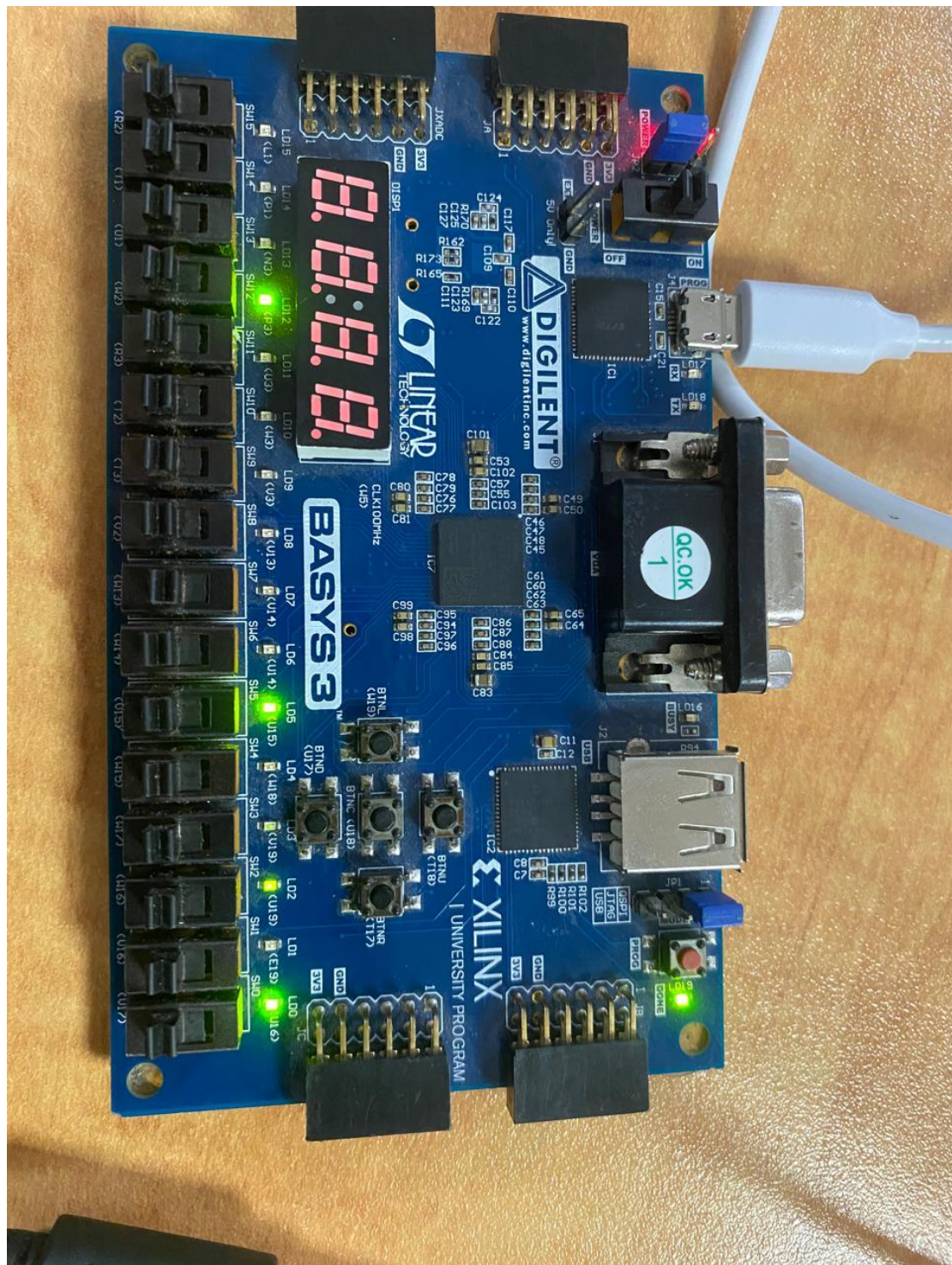




Xor

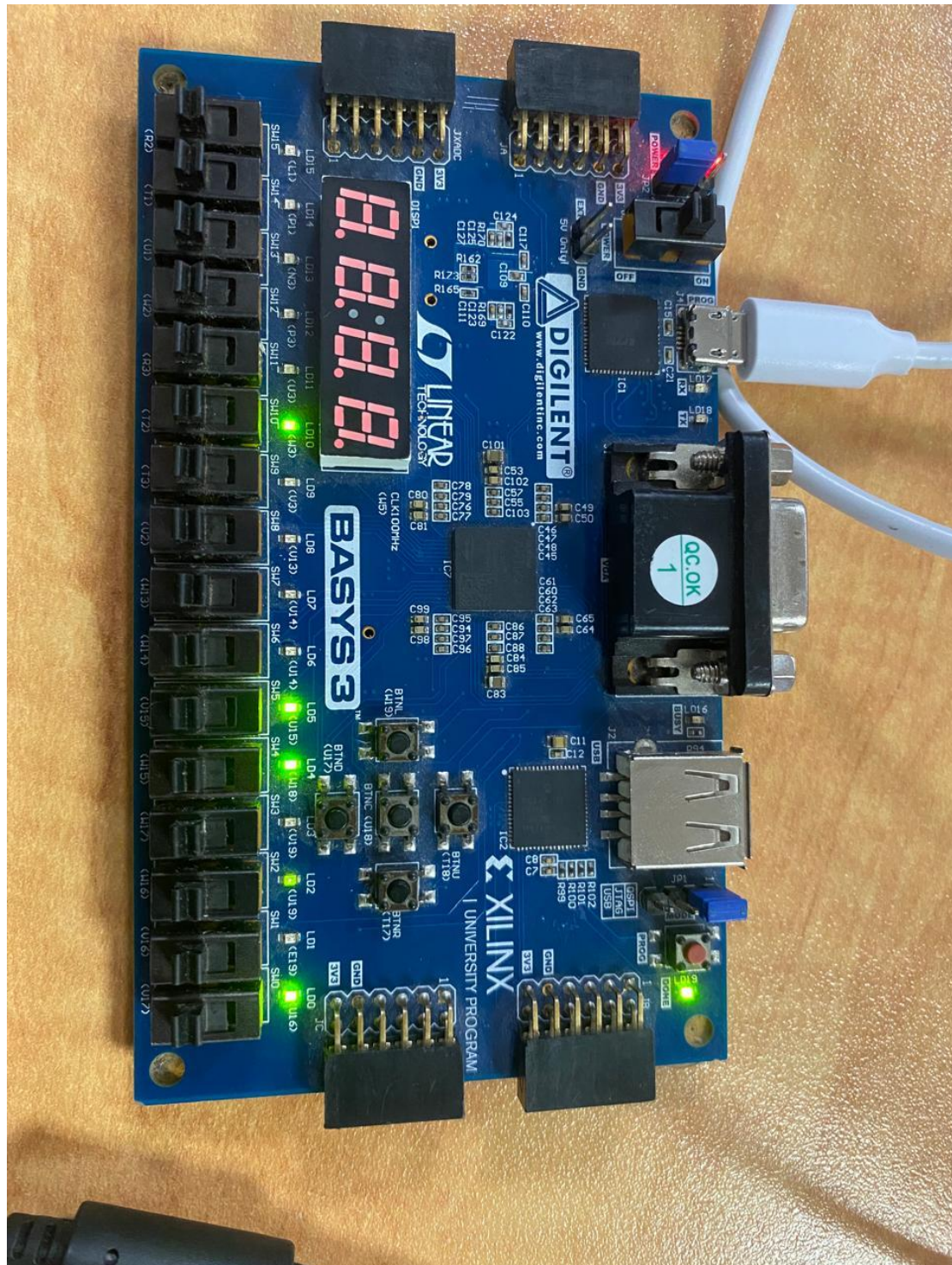


SII



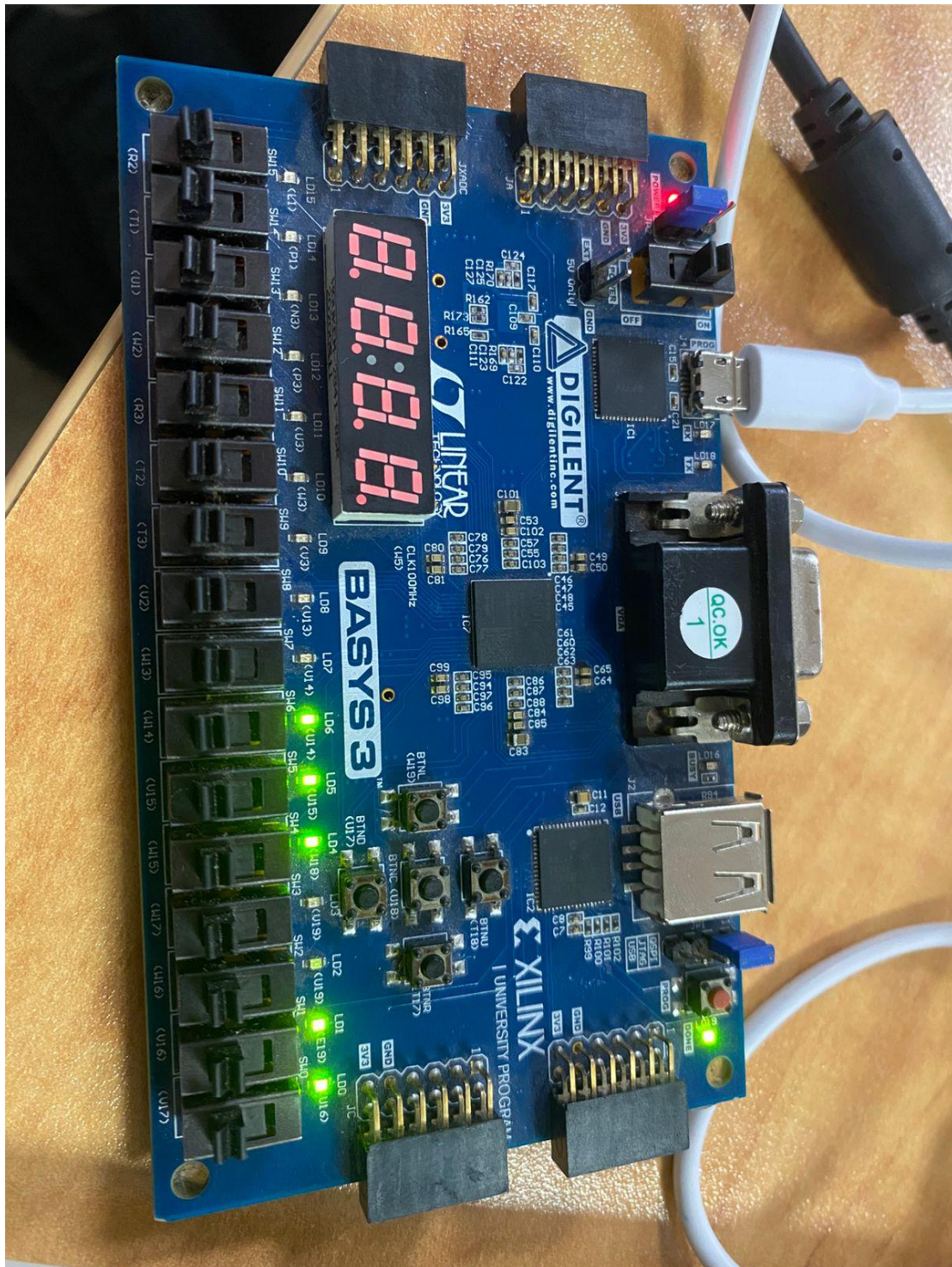


Srl





Beq





Lab 07

Points Distribution

Task No.	LR2 Code	LR 5 Results	LR11 Design	AR7
Task 1	-	-	/20	/20
Task 2	/20	/15		
Task 3	/10	/15		
Total Points	/80			/20
CLO Mapped	CLO 1			

For description of different levels of the mapped rubrics, please refer the provided Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics.

#	Assessment Elements	Level 1: Unsatisfactory Points 0-1	Level 2: Developing Points 2	Level 3: Good Points 3	Level 4: Exemplary Points 4
LR2	Program/Code / Simulation Model/ Network Model	Program/code/simulation model does not implement the required functionality and has several errors. The student is not able to utilize even the basic tools of the software.	Program/code/simulation model/network model has some errors and does not produce completely accurate results. Student has limited command on the basic tools of the software.	Program/code/simulation model/network model gives correct output but not efficiently implemented or implemented by computationally complex routine.	Program/code/simulation /network model is efficiently implemented and gives correct output. Student has full command on the basic tools of the software.
LR5	Results & Plots	Figures/ graphs / tables are not developed or are poorly constructed with erroneous results. Titles, captions, units are not mentioned. Data is presented in an obscure manner.	Figures, graphs and tables are drawn but contain errors. Titles, captions, units are not accurate. Data presentation is not too clear.	All figures, graphs, tables are correctly drawn but contain minor errors or some of the details are missing.	Figures / graphs / tables are correctly drawn and appropriate titles/captions and proper units are mentioned. Data presentation is systematic.
AR9	Report	All the in-lab tasks are not included in report and / or the report is submitted too late.	Most of the tasks are included in report but are not well explained. All the necessary figures / plots are not included. Report is submitted after due date.	Good summary of most the in-lab tasks is included in report. The work is supported by figures and plots with explanations. The report is submitted timely.	Detailed summary of the in-lab tasks is provided. All tasks are included and explained well. Data is presented clearly including all the necessary figures, plots and tables.